

Chat App Frontend UI

Introduction

This project is a **Chat Application Frontend UI** built using **Streamlit** in Python.

The goal of this project is to design a simple, interactive, and user-friendly chat interface that allows:

- Creating new chats
- Renaming, archiving, deleting, or sharing chats
- Displaying messages in a clean format
- Taking user input through a chat input field

```
chatapp.py > ...  
import streamlit as st  
import uuid  
import datetime
```

We Import These Libraries

- **import streamlit as st** → Used to build the entire frontend (UI components like sidebar, buttons, chat messages, input fields).
- **import uuid** → Generates unique IDs for each chat so that no two chats clash.
- **import datetime** → Stores the timestamp when a chat is created.

Without these imports:

- The app would not have a UI (streamlit).
- Chats could not be uniquely identified (uuid).
- Chats would not have creation dates (datetime).

Code Explanation

```
st.set_page_config(page_title="Chat App", layout="wide")
```

This defines the **page title** and layout to ensure a wide, clean UI.

Helpers

```
# -----  
def now_iso():  
    return datetime.datetime.now().isoformat()  
  
def make_chat(title="New Chat"):  
    return {  
        "id": uuid.uuid4().hex,  
        "title": title,  
        "messages": [],  
        "archived": False,  
        "created_at": now_iso(),  
    }  
  
def get_chat(cid):  
    for c in st.session_state["chats"]:  
        if c["id"] == cid:  
            return c  
    return None
```

now_iso()

- This function fetches the **current system date and time** using Python's datetime module.
- The result is converted into **ISO 8601 format** (a standard format for timestamps).
- Example: 2025-09-24T12:30:45.123456
- **Purpose:** Provides accurate timestamps for when chats are created, which helps in **sorting chats** and maintaining a timeline.

make_chat()

- This function is responsible for **creating a new chat object** whenever the user starts a new chat.
- A chat object is like a small **data container** that stores:
 - id → a unique identifier (using uuid).
 - title → the name of the chat (default: “New Chat”).
 - messages → an empty list where all messages will be saved.
 - archived → a status flag (active or archived).
 - created_at → timestamp of when the chat was created (using now_iso()).
- **Purpose:** Ensures each chat is unique and ready to store user inputs and actions.

get_chat(cid)

- This function retrieves a **specific chat object** from all the available chats using its **unique chat ID (cid)**.
- It loops through the list of chats saved in st.session_state["chats"].
- If it finds a matching ID, it returns that chat; if not, it returns None.
- **Purpose:** Allows the app to display and update the **currently selected chat** without affecting others.

Session State Initialization

```

if "chats" not in st.session_state:
    st.session_state["chats"] = []

if "current_chat" not in st.session_state:
    new_chat = make_chat()
    st.session_state["chats"].insert(0, new_chat)
    st.session_state["current_chat"] = new_chat["id"]

if "show_menu" not in st.session_state:
    st.session_state["show_menu"] = None

if "rename_chat" not in st.session_state:
    st.session_state["rename_chat"] = None

```

Streamlit does not store variables permanently, so `st.session_state` is used to store chats, current chat, and menu visibility.

```

# -----
# Sidebar (ChatAPP)
# -----
with st.sidebar:
    st.markdown("<h2 style='margin-bottom:15px;'>Chat APP</h2>", unsafe_allow_html=True)

    # + New Chat button
    if st.button("+ New Chat", use_container_width=True):
        new_chat = make_chat()
        st.session_state["chats"].insert(0, new_chat)
        st.session_state["current_chat"] = new_chat["id"]
        st.rerun()

```

Sidebar – Chat Management

1.with st.sidebar:

- This opens Streamlit sidebar section (the left panel of the app).
- All code inside this block will appear inside the sidebar.

2.Create a New Chat

Allows users to start a fresh conversation with a unique ID and empty message list.

This ensures multiple chats can be handled independently.

st.rerun()?

- In Streamlit, the entire script runs **from top to bottom** every time you interact with the app (like clicking a button).
- But sometimes, after making changes to st.session_state, you need the app to **refresh immediately** to reflect the updates.

```
# Search bar
search_query = st.text_input("Search chats", placeholder="🔍 Search chats")

# Filter chats
chats = sorted(st.session_state["chats"], key=lambda x: x["created_at"], reverse=True)
chats = [
    c for c in chats
    if (not c["archived"]) and (search_query.lower() in c["title"].lower())
]

st.markdown("<h4 style='margin-top:20px;'>Chats</h4>", unsafe_allow_html=True)

for chat in chats:
    col1, col2 = st.columns([8, 1])
    with col1:
        if st.button(
            chat["title"],
            key=f"chat_{chat['id']}",
            help="Open chat",
            use_container_width=True
        ):
            st.session_state["current_chat"] = chat["id"]
            st.rerun()

    with col2:
        if st.button(":", key=f"menu_{chat['id']}"):
            st.session_state["show_menu"] = (
                None if st.session_state["show_menu"] == chat["id"] else chat["id"]
            )
```

1.Search Bar

Provides a search bar to quickly filter chats by their titles.

Helps users find specific chats without scrolling through the entire list.

2.Open Any Chat(filter)

Users can select and open any chat from the sidebar to continue their conversation. The selected chat's messages are displayed in the main chat area.

```
with st.container():
    st.markdown("<div class='menu-box'>", unsafe_allow_html=True)

    if st.button("Share", key=f"share_{chat['id']}"):
        st.info(f"Share placeholder for '{chat['title']}'")
        st.session_state["show_menu"] = None

    if st.button("Rename", key=f"rename_{chat['id']}"):
        st.session_state["rename_chat"] = chat["id"]
        st.session_state["show_menu"] = None
        st.rerun()

    if st.button("Archive", key=f"archive_{chat['id']}"):
        chat["archived"] = True
        st.session_state["show_menu"] = None
        st.rerun()

    if st.button("Delete", key=f"delete_{chat['id']}"):
        st.session_state["chats"] = [
            c for c in st.session_state["chats"] if c["id"] != chat["id"]
        ]
        if st.session_state["current_chat"] == chat["id"]:
            st.session_state["current_chat"] = (
                st.session_state["chats"][0]["id"] if st.session_state["chats"] else None
            )
        st.session_state["show_menu"] = None
        st.rerun()

st.markdown("</div>", unsafe_allow_html=True)
```

4. Menu Button (:)

Perform Actions (Rename, Archive, Share, Delete)

- **Rename** → Change the chat's title for easy identification.
- **Archive** → Hide inactive chats while keeping them stored safely.
- **Share** → Placeholder for future sharing/export options.
- **Delete** → Permanently remove a chat and its messages.

```

# -----
# Main Chat Area
# -----
current_chat = get_chat(st.session_state["current_chat"])

if current_chat:
    if not current_chat["messages"]:
        st.markdown(
            """
            <div style="text-align:center; margin-top:120px;">
            |   <h2>What's on your mind?</h2>
            </div>
            """,
            unsafe_allow_html=True
        )
    else:
        for msg in current_chat["messages"]:
            with st.chat_message(msg["role"]):
                st.markdown(msg["content"])

```

Main Chat Area

- The **Main Chat Area** is the **central part of the app** where the actual conversation is displayed.
- It shows all the **messages** from the **currently selected chat** (based on `st.session_state["current_chat"]`).
- ◆ If the chat has messages:
 - The app loops through each message and displays it using **`st.chat_message()`**.
 - Messages are shown in a **clean, role-based format** (e.g., "user" on one side, "assistant" on the other).
 - This makes the conversation look like a **real chat interface**, similar to WhatsApp or ChatGPT.
- ◆ If the chat is empty:
 - Instead of showing nothing, the app displays a **placeholder message**:
 - What's on your mind?
 - This acts as a **friendly prompt** for the user to start typing their first message.
 - It improves the UI and avoids a blank screen.

```

# -----
# Chat input
# -----
if prompt := st.chat_input("Type here"):
    if current_chat and current_chat["title"] == "New Chat" and not current_chat["messages"]:
        current_chat["title"] = prompt[:30] + "..."

    current_chat["messages"].append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

```

Chat Input (User Message Box)

- At the bottom of the app, there is a **chat input field** created using:
- `prompt = st.chat_input("Type here")`
- This input box is where the **user types their messages**.
- Once the user presses **Enter** (or sends the message):
 1. The typed message is captured in the variable `prompt`.
 2. The message is immediately displayed in the **Main Chat Area** under the role "user".
 3. The message is also **saved inside the current chat's messages list** in `st.session_state`.
 4. If the chat is brand new and still named "New Chat", its title is automatically updated to the first message text.

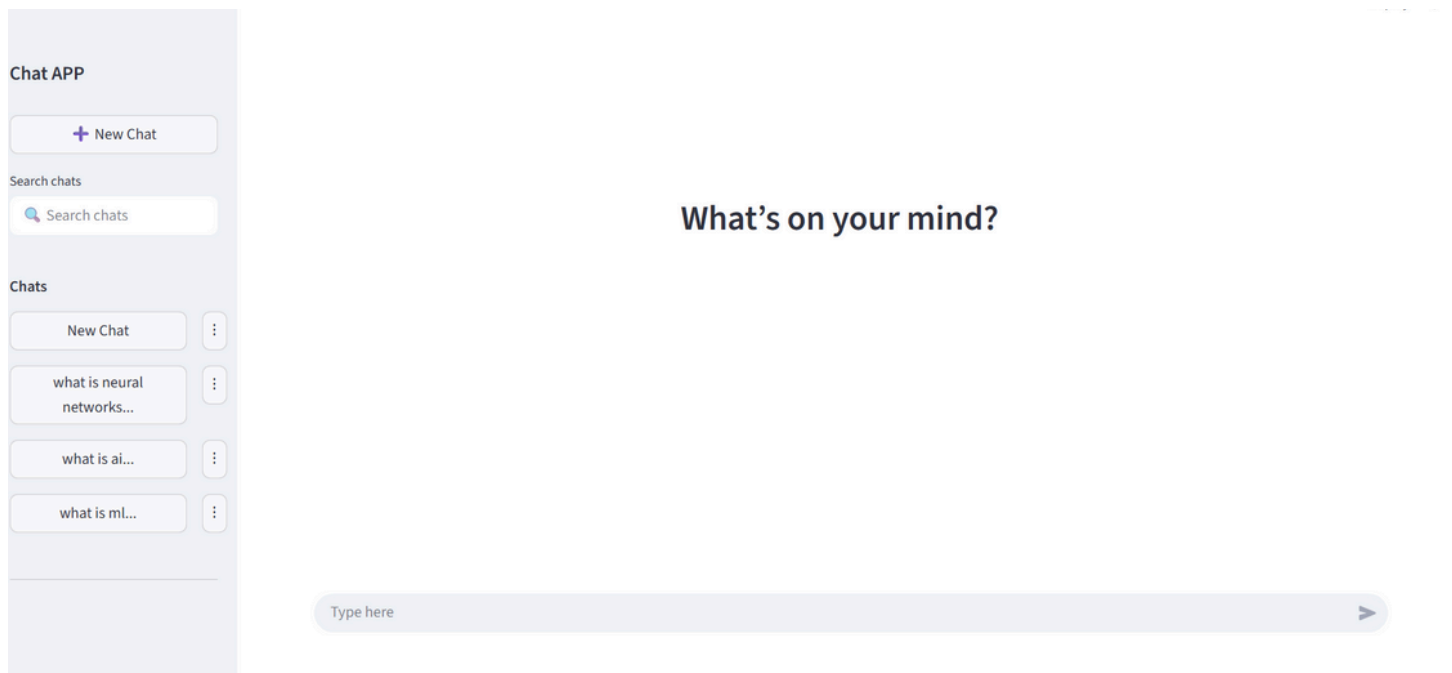
Options & Features Implemented

- 1 **Create New Chat** – adds a blank chat.
- 2 **Search Chat** – filters chats by name.
- 3 **Rename Chat** – edit the chat title.
- 4 **Archive Chat** – hides chat from main list.
- 5 **Delete Chat** – removes permanently.
- 6 **Share Option (Placeholder)** – shows info message.
- 7 **Chat Input & Messages** – send messages like a real chat.

6. Errors Faced & Fixes

Error Faced	Reason	Fix
State reset on refresh	Streamlit reruns code on every change	Used st.session_state to persist data
Chats not updating immediately	Streamlit needs rerun	Added st.rerun() after actions
Empty chat title	If user didn't rename	Auto-title: first message text
Style not applying	HTML + CSS conflicts	Used unsafe_allow_html=True

Output



A **functional chat app UI** with sidebar and chat area.

Messages appear like WhatsApp/ChatGPT style (role-based).

Clean dropdown menu for managing chats.

Future Scope & Features to Add

AI Integration – connect with GPT API to generate responses.

User Authentication – login/signup with username.

Export/Import Chats – download chat history as PDF/CSV.

Dark Mode Support – toggle themes.

Multi-user Chat – allow multiple users to chat in real-time.

Conclusion

This project demonstrates how **Python + Streamlit** can be used to create a **modern chat frontend UI** with state management, search functionality, and CRUD operations.

It helped me learn about:

- Streamlit's **session state**
- UI design using **sidebar & containers**
- Handling **user input dynamically**

The app currently works as a **frontend-only prototype**, but with future enhancements, it can evolve into a **fully functional chat system**.

THANK YOU